

Aula 4 — E2E com Detox

"O app real, no simulador, sem sleeps."

Gray-box testing · sync automático · Robot Pattern

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 29/06/2026

Recap da Aula 3

- **Setup do projeto** RN com Jest + RNTL configurados
- **Domain puro:** `posterUrl`, `isTokenError`, `favoritesStore` com Jest
- **Componentes RNTL:** `MovieCard` — render, toque, navegação mockada
- **Integração:** `NavigationContainer`, `QueryClientProvider`, `renderHook`
- Diferença unit × integração × E2E na pirâmide

Atividade 2 (suíte unitária, até 21/06) — entregue?

Agenda da aula (3h30)

1. **Bloco 1 (75min)** — Detox: conceito, setup, gray-box sync, matchers
2. **Intervalo (30min)**
3. **Bloco 2 (75min)** — Robot Pattern + 3 flows hands-on + CI
4. **Esquenta (15min)** — Atividade 3 + prévia da Aula 5

Objetivos de aprendizagem

- **Entender** o que é gray-box testing e por que Detox não usa sleeps
- **Configurar** Detox num projeto React Native existente
- **Implementar** 3 flows E2E: autenticação, navegação entre telas, favoritos
- **Aplicar** Robot Pattern para desacoplar localização de elemento da lógica do teste
- **Integrar** Detox ao CI (GitHub Actions) com simulador headless

RNTL (Aula 3) vs Detox — onde estamos na pirâmide

E2E – Detox	← Hoje	app compilado + simulador real
Integração – RNTL	← A3	React renderizado em memória
Unitários – Jest	← A3	Node puro, sem React

RNTL: rápido, sem simulador, mas sem a navegação real nem o runtime nativo.

Detox: simula o usuário de verdade — gestos, permissões, deeplinks, push.

O que é Detox?

Framework de testes E2E **gray-box** para React Native desenvolvido pela Wix.

Gray-box = o test runner pode se comunicar com o JS thread do app para sincronizar estado — não é black-box cego.

Benefícios diretos do gray-box:

- **Sem sleeps** — sabe quando o app terminou de renderizar
- **Sem flake por animação** — Idle Resources / Looper hooks
- **Sem `waitFor` cego** — espera o app ficar ocioso, não um timeout

Diferença do que vem antes: RNTL mocka o runtime. Detox roda o app compilado de verdade no simulador.

Arquitetura Detox



O Test Server sabe quando o JS thread está **idle** (sem JS em execução, sem animações) antes de enviar o próximo comando.

Setup Detox — projeto existente

```
npm install --save-dev detox @config-plugins/detox
```

```
# iOS — necessário Xcode  
cd ios && pod install
```

```
// .detoxrc.js  
module.exports = {  
  testRunner: {  
    args: { '$0': 'jest', config: 'e2e/jest.config.js' },  
  },  
  apps: {  
    'ios.debug': {  
      type: 'ios.app',  
      binaryPath: 'ios/build/Build/Products/Debug-iphonesimulator/MeuApp.app',  
      build: 'xcodebuild -workspace ios/MeuApp.xcworkspace -scheme MeuApp -sdk iphonesimulator -derivedDataPath ios/build',  
    },  
  },  
  devices: {  
    simulator: {  
      type: 'ios.simulator',  
      device: { type: 'iPhone 15' },  
    },  
  },  
  configurations: {  
    'ios.sim.debug': {  
      device: 'simulator',  
      app: 'ios.debug',  
    },  
  },  
}
```


Primeiro build + run

```
# Build o app pra simulador (uma vez, ou quando mudar código nativo)
npx detox build --configuration ios.sim.debug

# Rodar todos os testes E2E
npx detox test --configuration ios.sim.debug

# Rodar só um arquivo
npx detox test --configuration ios.sim.debug e2e/login.test.ts
```

O simulador abre automaticamente. Primeira vez demora pelo build.

testID — contrato entre app e teste

```
// No componente React Native
<TouchableOpacity testID="login-submit-button">
  <Text>Entrar</Text>
</TouchableOpacity>

<TextInput
  testID="login-email-input"
  placeholder="Email"
/>
```

```
// No teste Detox
await element(by.id('login-submit-button')).tap();
await element(by.id('login-email-input')).typeText('user@email.com');
```

Convenção adotada neste curso: tela-elemento-acao — ex: login-email-input, home-fav-button.

Matchers — como o Detox encontra elementos

```
by.id('testID')           // ← preferido (estável)
by.text('Entrar')         // frágil a mudança de copy
by.label('Entrar')        // accessibilityLabel
by.type('RCTTextInput')    // por tipo de componente nativo
by.traits(['button'])      // por accessibilityTraits (iOS)
```

Mesma hierarquia da RNTL: prefira `id` para testes E2E. `text` é escape hatch.

Ações Detox — o que você pode fazer

```
// Toques
await element(by.id('submit-btn')).tap();
await element(by.id('submit-btn')).longPress();
await element(by.id('submit-btn')).multiTap(2);

// Texto
await element(by.id('email-input')).typeText('user@test.com');
await element(by.id('email-input')).clearText();
await element(by.id('email-input')).replaceText('novo@test.com');

// Scroll
await element(by.id('movie-list')).scroll(300, 'down');
await element(by.id('movie-list')).scrollTo('bottom');

// Swipe
await element(by.id('card')).swipe('left', 'fast', 0.75);

// Keyboard
await element(by.id('search')).tapReturnKey();
```

Asserções Detox

```
// Visibilidade
await expect(element(by.id('welcome-text'))).toBeVisible();
await expect(element(by.id('loading'))).not.toBeVisible();

// Texto
await expect(element(by.id('title'))).toHaveText('Matrix');
await expect(element(by.id('count'))).toHaveText('3');

// Valor (inputs)
await expect(element(by.id('email-input'))).toHaveValue('user@test.com');

// Existência na árvore (não necessariamente visível)
await expect(element(by.id('hidden-modal'))).toExist();
await expect(element(by.id('removed'))).not.toExist();
```

waitFor — quando o sync não basta

Detox sincroniza automaticamente. `waitFor` é para casos explícitos:

```
// Aguarda elemento aparecer (ex: após fetch de rede mockado)
await waitFor(element(by.id('movie-list')))
  .toBeVisible()
  .withTimeout(5000);

// Aguarda com scroll (lazy loading)
await waitFor(element(by.id('movie-item-42')))
  .toBeVisible()
  .whileElement(by.id('movie-list'))
  .scroll(100, 'down');
```

`waitFor` com `withTimeout(5000)` não é sleep — o Detox verifica ativamente. Mas evite quando o sync automático já cobre.

Primeiro teste completo

```
// e2e/login.test.ts
import { device, element, by, expect } from 'detox';

describe('Login', () => {
  beforeAll(async () => {
    await device.launchApp();
  });

  beforeEach(async () => {
    await device.reloadReactNative();
  });

  it('usuário faz login com credenciais válidas', async () => {
    await element(by.id('login-email-input')).typeText('user@test.com');
    await element(by.id('login-password-input')).typeText('secret123');
    await element(by.id('login-submit-button')).tap();

    await expect(element(by.id('home-screen'))).toBeVisible();
    await expect(element(by.id('welcome-text'))).toHaveText('Olá, user!');
  });
});
```

Robot Pattern — por que usar?

Problema: testes E2E sem abstração acoplam localização (testID) ao fluxo do teste.

```
// ❌ Sem Robot — qualquer renomeio de testID quebra 12 testes  
await element(by.id('login-email-input')).typeText('user@test.com');  
await element(by.id('login-password-input')).typeText('secret');  
await element(by.id('login-submit-button')).tap();  
await element(by.id('login-submit-button')).tap(); // duplicado em outro teste
```

```
// ✅ Com Robot — o testID muda em 1 lugar, os testes ficam legíveis  
await LoginRobot.fillEmail('user@test.com');  
await LoginRobot.fillPassword('secret');  
await LoginRobot.submit();
```

Wharton, J. (2015) — *Instrumentation Testing Robots*. Mesmo princípio que Page Object Model mas para mobile. Cada tela tem um Robot.

Robot Pattern — implementação

```
// e2e/robots/LoginRobot.ts
export const LoginRobot = {
  async fillEmail(email: string) {
    await element(by.id('login-email-input')).typeText(email);
    return LoginRobot;
  },
  async fillPassword(password: string) {
    await element(by.id('login-password-input')).typeText(password);
    return LoginRobot;
  },
  async submit() {
    await element(by.id('login-submit-button')).tap();
    return HomeRobot; // retorna o Robot da próxima tela
  },
};

// e2e/login.test.ts - limpo e legível
it('faz login com credenciais válidas', async () => {
  await LoginRobot
    .fillEmail('user@test.com')
    .then(r => r.fillPassword('secret'))
    .then(r => r.submit());
  await expect(element(by.id('home-screen'))).toBeVisible();
});
```

Robot Pattern — benefícios reais

Sem Robot	Com Robot
testID espalhado em todos os testes	testID centralizado no Robot
Renomear testID = <code>grep + sed</code> em N arquivos	Renomear testID = 1 linha no Robot
Teste lê como HTML	Teste lê como especificação
Difícil de reutilizar setup	<code>HomeRobot.searchFor('Matrix')</code> em qualquer teste

Qualquer E2E com > 5 testes se beneficia do Robot Pattern. Sem isso, manutenção escala linearmente com a suíte.

Detox — mockando rede

Detox não mocka rede por padrão — app faz requests reais. Para testes estáveis:

Opção 1: variável de ambiente no launchApp

```
await device.launchApp({ launchArgs: { detoxURLBlacklistRegex: '.*' } });
```

Opção 2: servidor mock local (MSW ounock)

```
// setup.ts
import { setupServer } from 'msw/node';
const server = setupServer(
  rest.get('*/movie/popular', (_, res, ctx) =>
    res(ctx.json({ results: [{ id: 1, title: 'Matrix' }] }))),
);
beforeAll(() => server.listen());
afterAll(() => server.close());
```

MSW + Detox é o padrão mais limpo para RN: mock de rede sem alterar a lógica de negócio.

Roteiro hands-on Bloco 1 (75min)

App alvo: mesmo app do catálogo de filmes das aulas anteriores.

1. **Setup** — `.detoxrc.js` + `npx detox build` — **15min**
2. **Flow 1: login** — email + senha + assert tela home — **15min** ★
3. **Flow 2: busca** — digita + scroll + assert resultado — **15min**
4. **Flow 3: favoritar** — tap coração + assert contador — **15min**
5. Adicionar `beforeEach: device.reloadReactNative()` e discutir isolamento — **5min**
6. Resolver 1 flake que você vai criar intencionalmente — **10min**

Roteiro hands-on Bloco 2 (75min)

1. **Robot Pattern** — extrair LoginRobot + HomeRobot dos 3 flows — **20min** ★
2. **Flow 4: erro de login** — credenciais erradas → mensagem de erro — **15min**
3. **Flow 5: navegação deep** — favoritar → abrir lista favoritos → remover — **20min**
4. Rodar `npx detox test` completo → todos verdes — **10min**
5. Discussão: O que foi mais difícil? Onde o sync automático não bastou? — **10min**

Detox no CI (GitHub Actions)

```
name: E2E iOS

on: [push]

jobs:
  e2e:
    runs-on: macos-latest # necessário pra Xcode + iOS Simulator
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22' }
      - run: npm install
      - run: cd ios && pod install
      - name: Build Detox
        run: npx detox build --configuration ios.sim.debug
      - name: Run E2E
        run: npx detox test --configuration ios.sim.debug --headless
```

`macos-latest` custa ~10x mais que `ubuntu-latest`. Roda E2E em PRs, não em todo push.

Pitfalls Detox

- ✗ `sleep(2000)` no teste — o sync automático é a vantagem do Detox; usar sleep derrota isso
- ✗ `testID` em texto, não em ID — frágil a tradução e A/B test
- ✗ Um `beforeAll` sem `reloadReactNative` — estado vazando entre testes → flake de ordem
- ✗ Rodar no simulador que estava aberto — Detox gerencia o simulador; não abra manualmente antes
- ✗ Testar lógica de negócio em E2E — isso é do Jest/RNLT. E2E prova o fluxo de usuário, não a função
- ✗ Sem Robot Pattern em suíte grande — manutenção escala mal

Atividade 3 — Detox E2E (15 pts)

Entrega: até 28/06.

#	Entrega	Pontos
1	3 flows Detox rodando no simulador iOS ou emulador Android	3
2	<code>device.reloadReactNative()</code> em <code>beforeEach</code> (isolamento)	1
3	Robot Pattern implementado para ao menos 1 tela	3
4	Flow de erro (credencial inválida → mensagem aparece)	2
5	Flow de estado persistido (favoritar → verificar em outra tela)	3
6	Screenshot de suíte verde (<code>detox test</code> sem falhas)	1
7	Breve análise (até 1pg): o que o RNTL não conseguiria testar aqui?	2
Bônus (+2)	CI configurado (GitHub Actions rodando <code>detox test</code>)	+2

Leituras obrigatórias (pré-Aula 5)

- **Wix Engineering** — *Detox: Gray Box E2E Testing Framework for Mobile Apps*
- **Wharton, J. (2015)** — *Instrumentation Testing Robots*
- **iDFlakies (Lam et al., 2019)** — causas de flakiness em E2E mobile
- **OWASP MASVS v2.1** — preparação Aula 5 (segurança mobile)

Próxima aula (29/06)

Aula 5 — Performance, Segurança Mobile e Observabilidade

Pré-requisito:

- Atividade 3 entregue
- Android Studio Profiler funcionando **ou** Xcode Instruments funcionando
- MobSF (Docker) configurado

Obrigado!

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith

Próxima segunda, 15/06, 19:00